

คู่มือการใช้งาน SonarQube

สำหรับการสอนวิเคราะห์คุณภาพและความปลอดภัยของโค้ด

ตัวอย่างระบบ: OWASP Juice Shop

จัดทำโดย อาจารย์ ดร. สุริยะ พินิจการ

ภาควิชาเทคโนโลยีสารสนเทศและการสื่อสาร คณะศิลปศาสตร์และวิทยาศาสตร์ มหาวิทยาลัยเกษตรศาสตร์ วิทยาเขตกำแพงแสน

สารบัญ

1. ภาพรวมและวัตถุประสงค์
2. การติดตั้ง SonarQube ด้วย Docker
3. การเตรียมระบบตัวอย่าง (OWASP Juice Shop)
4. การสร้าง Token
5. การสแกนโค้ดด้วย SonarScanner
6. การอ่านผลรายงานบนเว็บ SonarQube
7. การส่งออกรายงานเป็นไฟล์ PDF
8. การแก้ปัญหาที่พบบ่อย (Troubleshooting)
9. แนวทางการใช้สอนในคาบเรียน

1. ภาพรวมและวัตถุประสงค์

SonarQube เป็นเครื่องมือวิเคราะห์คุณภาพและความปลอดภัยของโค้ด (Static Code Analysis) ที่ใช้กันแพร่หลายในอุตสาหกรรม
คู่มือนี้จัดทำขึ้นเพื่อใช้สอนนักศึกษาให้เข้าใจแนวคิด Code Quality, Security Vulnerabilities และ Quality Gate
โดยใช้ระบบตัวอย่างที่มีช่องโหว่ให้เห็นชัดเจน คือ OWASP Juice Shop

สิ่งที่นักศึกษจะได้เรียนรู้

- ติดตั้งและใช้งาน SonarQube Community Edition ผ่าน Docker
- สแกนโค้ดจริงด้วย SonarScanner CLI
- อ่านและตีความผลรายงาน (Bugs, Vulnerabilities, Code Smells, Coverage)
- เข้าใจแนวคิด Quality Gate และ Technical Debt
- ส่งออกผลการวิเคราะห์เป็นรายงาน PDF ผ่าน SonarQube Web API

2. การติดตั้ง SonarQube ด้วย Docker

รัน SonarQube Community Edition ด้วยคำสั่งเดียว:

```
docker run -d --name sonarqube -p 9000:9000 sonarqube:community
```

จากนั้นเปิดเบราว์เซอร์ไปที่ <http://localhost:9000>

- Login เริ่มต้น: admin / admin
- ระบบจะบังคับให้เปลี่ยนรหัสผ่านทันทีในการเข้าใช้ครั้งแรก

หมายเหตุ: ไม่ต้องสร้าง Project, ตั้งค่า Quality Gate หรือ Quality Profile เอง — ใช้ค่า default ("Sonar way") ได้เลยสำหรับการสอน

3. การเตรียมระบบตัวอย่าง (OWASP Juice Shop)

OWASP Juice Shop เป็นเว็บแอปพลิเคชัน (Node.js/TypeScript + Angular) ที่ถูกออกแบบมาให้มีช่องโหว่ด้านความปลอดภัย
จำนวนมากโดยเจตนา เหมาะสำหรับสอนในหมวด Vulnerabilities และ Security Hotspots

```
git clone https://github.com/juice-shop/juice-shop.git
cd juice-shop
```

ไฟล์ตั้งค่า sonar-project.properties

สร้างไฟล์นี้ไว้ที่ root ของโฟลเดอร์ juice-shop:

```
sonar.projectKey=juice-shop-demo
sonar.projectName=Juice Shop Demo
sonar.sources=.
sonar.exclusions=node_modules/**,dist/**,build/**,frontend/dist/**
sonar.javascript.node.maxspace=4096
```

4. การสร้าง Token

Token ใช้สำหรับยืนยันตัวตนก่อนส่ง scan และดึงข้อมูลผ่าน API

1. คลิกไอคอนโปรไฟล์มุมขวาบน (วงกลมตัวอักษร A) → My Account
2. ไปที่แท็บ Security
3. ตั้งชื่อ token เช่น juice-shop-scan
4. เลือกประเภท Global Analysis Token
5. กด Generate แล้ว copy เก็บไว้ทันที (จะแสดงให้เห็นครั้งเดียวเท่านั้น)

หมายเหตุ: Token มีรูปแบบเช่น `squ_XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX` ระวังไม่ให้มีเว้นวรรคหรือสัญลักษณ์ `< >` ติดมาตอน copy

5. การสแกนโค้ดด้วย SonarScanner

รันคำสั่งนี้จากโฟลเดอร์ `juice-shop` (ที่มีไฟล์ `sonar-project.properties`):

```
docker run --rm \
  --memory=8g \
  -e SONAR_HOST_URL="http://host.docker.internal:9000" \
  -e SONAR_TOKEN="<TOKEN ที่ copy ไว้>" \
  -v "$(pwd):/usr/src" \
  sonarsource/sonar-scanner-cli
```

Scan สำเร็จจะเห็นข้อความท้าย log ว่า:

```
INFO EXECUTION SUCCESS
```

6. การอ่านผลรายงานบนเว็บ SonarQube

หลัง scan เสร็จ เข้าไปที่ Projects → `juice-shop-demo` จะเห็นหน้า Dashboard หลัก

6.1 Quality Gate

ป้ายสีเขียว (Passed) หรือสีแดง (Failed) แสดงผลรวมว่าโค้ดผ่านเกณฑ์คุณภาพขั้นต่ำหรือไม่

6.2 สี่หมวดหลัก

หมวด	ความหมาย	Rating
Reliability (Bugs)	โค้ดที่รันแล้วพังหรือทำงานผิดจากที่ตั้งใจ	A–E
Security (Vulnerabilities)	ช่องโหว่ที่โจมตีได้จริง เช่น SQL Injection, XSS	A–E
Security Review (Hotspots)	โค้ดที่น่าสงสัย ต้องให้คนตรวจสอบว่าเป็นปัญหาจริงหรือไม่	% reviewed
Maintainability (Code Smells)	โค้ดใช้งานได้ แต่แก้ไขหรืออ่านยากในอนาคต	A–E

Rating: A = ดีที่สุด ไปจนถึง E = แย่ที่สุด (เหมือนระบบเกรด)

6.3 การไล่ดู Issues

- คลิกที่ตัวเลขในแต่ละหมวด (เช่น "127 Vulnerabilities") เพื่อเข้าหน้า Issues
- ดู Severity: Blocker > Critical > Major > Minor > Info — ควรไล่ดูจาก Blocker ลงมาก่อน
- คลิกแต่ละ issue เพื่อดูตำแหน่งโค้ดจริงและคำอธิบาย "Why is this an issue?"

6.4 แนวคิดสำคัญอื่น ๆ

- Coverage (%) — สัดส่วนโค้ดที่มี unit test ครอบคลุม
- Duplications (%) — สัดส่วนโค้ดที่ซ้ำกันในโปรเจกต์
- New Code vs Overall Code — SonarQube เน้นวัดคุณภาพของ "โค้ดใหม่" เป็นหลักในการทำงานจริง ซึ่งเป็น mindset สำคัญของทีมพัฒนา

7. การส่งออกรายงานเป็นไฟล์ PDF

SonarQube Community Edition ไม่มีปุ่ม Export PDF ในตัว (เป็นฟีเจอร์ของรุ่น Developer Edition ขึ้นไป) จึงใช้สคริปต์ `sonar_report.py` ดึงข้อมูลผ่าน Web API แล้วสร้างเป็น PDF แทน

7.1 การติดตั้ง

```
pip install requests reportlab
```

7.2 การรันสคริปต์

```
python3 sonar_report.py \
  --host http://localhost:9000 \
  --project juice-shop-demo \
  --token <TOKEN>
```

ผลลัพธ์: ได้ไฟล์ juice-shop-demo_report.pdf ในโฟลเดอร์เดียวกัน

7.3 ตัวเลือกเพิ่มเติม

Flag	ความหมาย	ค่า default
--output	ตั้งชื่อไฟล์ PDF เอง	<project>_report.pdf
--top	จำนวนตัวอย่าง issues ที่แสดงในรายงาน	15

7.4 เนื้อหาในรายงาน

- หัวรายงาน: ชื่อโปรเจกต์, server, วันที่เวลาที่สร้าง
- Quality Gate: แถบสีเขียว/แดงบอกผ่านหรือไม่ผ่าน
- ตารางตัวชี้วัดหลัก: Bugs, Vulnerabilities, Security Hotspots, Code Smells, Coverage, Duplicated Lines, Lines of Code, Rating
- ตาราง Issues แยกตาม Severity
- ตัวอย่าง Issues ที่ severity สูงสุด พร้อมไฟล์และคำอธิบายย่อ

8. การแก้ปัญหาที่พบบ่อย (Troubleshooting)

8.1 Bridge server is unresponsive / ปัญหาหน่วยความจำ

อาการ: JS/TS analysis ล้มเหลว พร้อมข้อความ "The bridge server is unresponsive"

สาเหตุ: Node.js bridge ที่ใช้วิเคราะห์ JS/TS มีหน่วยความจำไม่พอ (พบบ่อยเมื่อสแกน Juice Shop เพราะโปรเจกต์ใหญ่)

วิธีแก้:

- เพิ่ม property sonar.javascript.node.maxspace=4096 (หรือสูงกว่า) ในไฟล์ sonar-project.properties
- เพิ่ม --memory=8g ให้กับคำสั่ง docker run
- สำคัญที่สุดสำหรับ Mac: เพิ่ม Memory ให้ Docker Desktop เอง ที่ Settings → Resources → Memory (อย่างน้อย 8GB) เพราะ Docker Desktop มี VM จำกัด memory อีกชั้นหนึ่ง ต่อให้ตั้ง --memory ใน container ก็ไม่ช่วยถ้า VM เองไม่มีให้
- ถ้าเครื่อง RAM จำกัดจริง ให้ลด scope การสแกน โดย exclude โฟลเดอร์ frontend ออกไปก่อน

8.2 HTTP 401 Unauthorized

สาเหตุ: Token ผิด, พิมพ์ผิด,หมดอายุ หรือมีช่องว่างติดมาตอน copy

วิธีแก้:

- สร้าง Token ใหม่จาก My Account → Security
- ตรวจสอบว่าไม่มีเว้นวรรคหรือสัญลักษณ์ < > ติดมา
- ทดสอบ token ก่อนด้วยคำสั่ง: curl -u "<token>": http://localhost:9000/api/authentication/validate

8.3 คำสั่งใน terminal ไม่ทำงาน (command not found)

บรรทัดที่เป็นเนื้อหาไฟล์ (เช่น `sonar.javascript.node.maxspace=4096`) ต้องเพิ่มในไฟล์ `sonar-project.properties` เท่านั้น ห้ามพิมพ์ลง terminal ตรง ๆ

8.4 Invalid character encountered ... (warning)

เป็นแค่ WARNING ไม่ใช่ error — เกิดจากไฟล์รูปภาพ/วิดีโอ/ไฟล์ binary ที่ scanner พยายามอ่านเป็น text ไม่กระทบผลการวิเคราะห์ ไม่ต้องแก้ไข

9. แนวทางการใช้สอนในคาบเรียน

ลำดับการสอนที่แนะนำ:

6. เปิดหน้า Dashboard ดู Quality Gate ก่อน (ภาพรวม Pass/Fail)
7. ไปดู Vulnerabilities → Security Hotspots (จุดเด่นของ Juice Shop)
8. เปิดตัวอย่าง 2-3 เคส คลิกดูโค้ดจริง พร้อมอ่าน "Why is this an issue?" ร่วมกัน
9. ปิดท้ายด้วย Bugs และ Code Smells แบบเร็ว ๆ
10. เชื่อมโยงแนวคิด New Code vs Overall Code เข้ากับการทำงานจริงใน CI/CD
11. ให้นักศึกษารันสคริปต์ `sonar_report.py` เพื่อ export ผลเป็น PDF เก็บเป็นหลักฐานส่งงาน

ข้อควรระวัง

- Juice Shop ก่อนขึ้นหน้าสำหรับ scan สด ๆ ในคาบ หากเครื่องนักศึกษามี RAM จำกัด แนะนำ pre-scan ไว้ล่วงหน้าและใช้ผลจาก server กลางแทน
- ควรกำชับนักศึกษาว่านี่คือระบบจำลองเพื่อการศึกษาเท่านั้น ห้ามนำเทคนิคไปใช้กับระบบจริงที่ไม่ได้รับอนุญาต